APP

Procesor care implementează o structură paralelă pipeline de citire – interpretare – execuție pentru procesare superscalară.

Arătați îmbunătățirea performanței față de procesare scalară cu pipeline/fără pipeline, știind că un ciclu de instrucțiuni conține

→ citire necesită o singură perioadă de ceas
→ decodare necesită două perioade de ceas
→ execuție necesită trei perioade de ceas

} 200 instrucțiuni

O structură fără pipeline : $(1+2+3)*200 = 1200$ cicluri de ceas

F D1 D2 E1 E2 E3
F D1 D2 E1 E2 E3 .....

O structură pipeline scalară : $1+2+(200*3) = 603$ cicluri de ceas

F D1 D2 E1 E2 E3
F D1 D2 E1 E2 E3 .....

O structură pipeline superscalară cu două unități paralele :

$1+2+\left(\frac{200}{2}*3\right) = 303$ cicluri de ceas

F D1 D2 E1 E2 E3
F D1 D2 E1 E2 E3
F D1 D2 E1 E2 E3 ..... fără necesități

F D1 D2 E1 E2 E3
F D1 D2 E1 E2 E3
F D1 D2 E1 E2 E3 ..... fără secvențial

Algoritmi paralelă

- granularitate modul => cât de complex?
- control concurent => excluderi mutuale?
- mecanisme datelor => ordinea operațiilor?
- geometria comunicației => interacțiunea modulelor?
- complexitate algoritm => număr operații?

Arhitecturi paralele

- complexitate procesor
- mod de operare
- structura memoriei
- rețea de comutare
- număr de procesoare / dimensiunea memoriei

Amdahl : $V_p = f_{seq} + \frac{f_{par}}{p}$

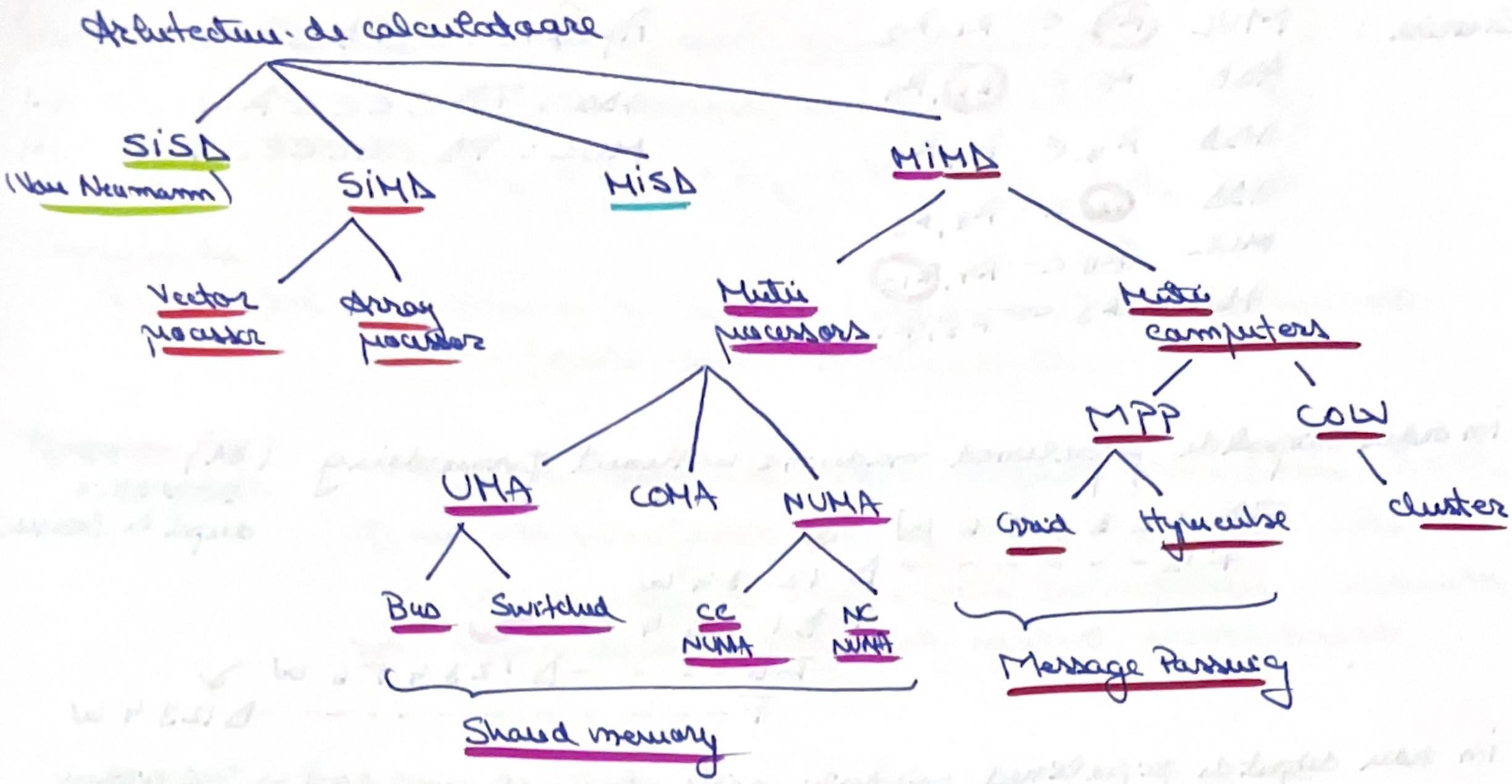
Limita de accelerare este determinată de gradul de secvențialitate, NU de numărul de procesoare!

Gustafson Barsis : $V_p = 1 + (p-1) f_{par}$

Problema de dimensiune mare

Sun and Ni : Introduce modelul bazat pe limita de memorie a unui procesor

Worton : Model ce aproximează operarea unui multiprocesor



Dataflow Graph

MUL $R3 \leftarrow R1, R2$

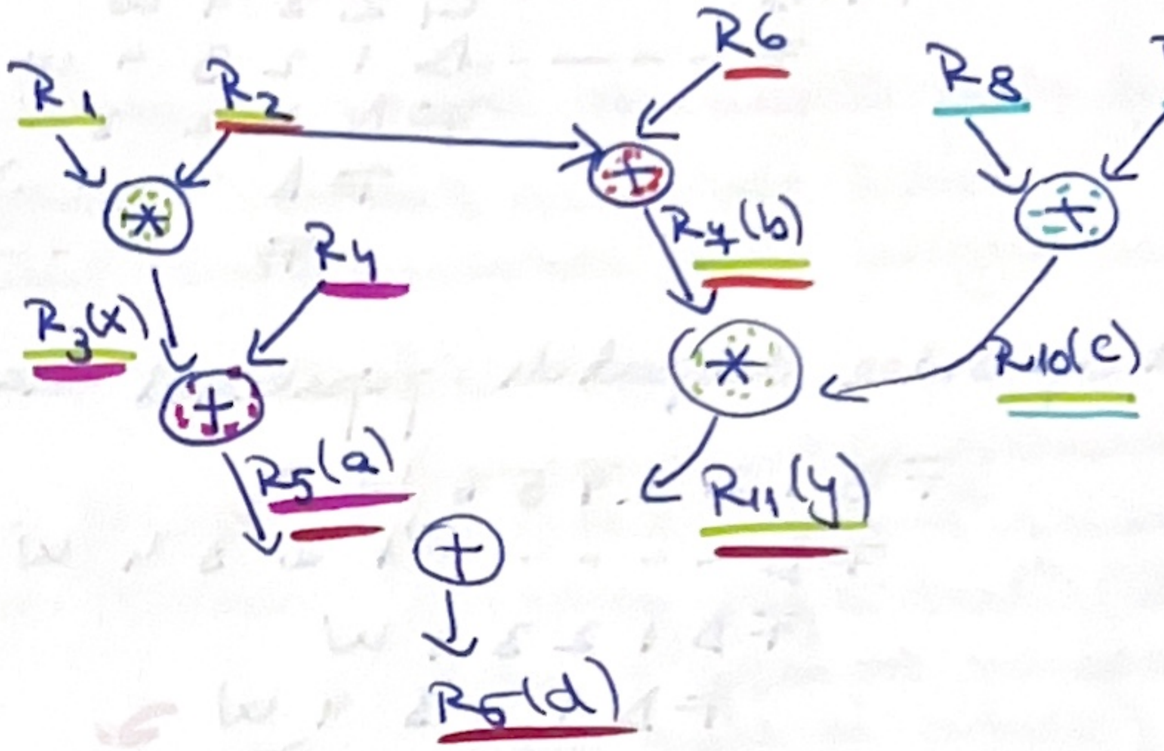
ADD $R5 \leftarrow R3, R4$

ADD $R7 \leftarrow R2, R6$

ADD $R10 \leftarrow R8, R9$

MUL $R11 \leftarrow R4, R10$

ADD $R5 \leftarrow R5, R11$



R(3) x

R(11) y

MUL val

R(5) ad → multiple val

R(4) b

R(10) c

AND val

Exercice :

MUL R3 ← R1, R2
ADD R5 ← R3, R4
ADD R4 ← R2, R6
ADD R8 ← R8, R9
MUL R11 ← R4, R10
ADD R5 ← R5, R11

Pipeline : FDERW

ADD : FD EEEERW (4)
MUL : FD EEEEEERW (6)

in order dispatch pipelined machine without forwarding (31) => attente à cause des dépendances

F D 1 2 3 4 5 6 W
F D - - - - - - D 1 2 3 4 W
F D 1 2 3 4 W
F D - - - - D 1 2 3 4 5 6 W
F - - - - - - - - - - - - D 1 2 3 4 W

in order dispatch pipelined machine with forwarding (25) => -6 cycles. Execute sub wait

F D 1 2 3 4 5 6 W
F D - - - - - E 1 2 3 4 W
F - - - - D 1 2 3 4 W
F D 1 2 3 4 W
F D - - - E 1 2 3 4 5 6 W
F - - - D - - - - E 1 2 3 4 W

Out of order dispatch pipelined machine with forwarding (20)
=> fait même, sauf le décode

F D 1 2 3 4 5 6 W
F D - - - - 1 2 3 4 W
F D 1 2 3 4 W
F D 1 2 3 4 W
F D - - - 1 2 3 4 5 6 W
F D - - - - - - - 1 2 3 4 W

Definiție: Spunem că S_i și S_j sunt **neinterferente** dacă:

* S_i este succesor sau predecesor al lui S_j
* $R_{S_i} \cap R_{S_j} = R_{S_i} \cap \Delta_{S_j} = \Delta_{S_i} \cap R_{S_j} = \emptyset$

Teorema de suficiență: Sistemele de sarcini formate din sarcini mutual neinterferente sunt **determinate**.

Teorema de necesitate: Fie C un sistem de sarcini astfel că pentru fiecare $S \subset S$, f_S nu este specificată, dar $\Delta_S \neq \emptyset$, $R_S \neq \emptyset$ sunt date.
C este determinat pentru orice interpretare a sarcinilor dacă numai dacă acestea sunt mutual neinterferente.

Supervizor → criterii de alocare: 1. FIFO : first in first out
2. SJF : shortest job first
3. ROUND ROBIN : rulare pe rotație

Contextul în care evoluează un sistem de sarcini concurente poate fi:
- centralizat (memorie comună accesibilă R/W)
- distribuit (memorie locală accesibilă unei singure sarcini)

⇒ excludere mutuală (ex: hardware → UCP & DMA)

În mediu centralizat:
- Algoritmul Dekker (cu două priorități → Stavăt-cu) (vector de boolene^2 → cine vrea?)
- Algoritmul Petterson (se adaug flag și: turn^{2+}) (LOAD STORE atomice) (nu se așteaptă niciodată mai mult de o tură) (ciclează cu priorități)

Excludere mutuală cu instrucțiunea XCHG:
→ XCHG (r, m) interschimbă, ca o operație atomică, conținutul registrului r cu celula de memorie m (semafor)
· dacă r_i este o variabilă locală a sarcinii S_i și e inițializată cu 0
· m este o variabilă globală, comună tuturor sarcinilor care e inițializată cu 1
→ e rezidual fără sau nu (se anulează cu cont exchange constant)
→ cazul general $m = l$ intră în secțiune critică (ca și poze)
→ mi se dă cazul unui e rezidual

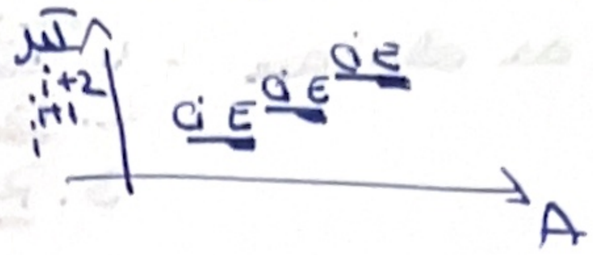
Excludere mutuală cu instrucțiunea TAS:
→ TAS se execută atomic, ca poze verifică m dacă e 0 intră și schimbă $m = 1$
→ trebuie să se verifice dacă e liber

Excludere cu LOCK & UNLOCK:
→ blocuri se delimitează zona respectivă de memorie TSB

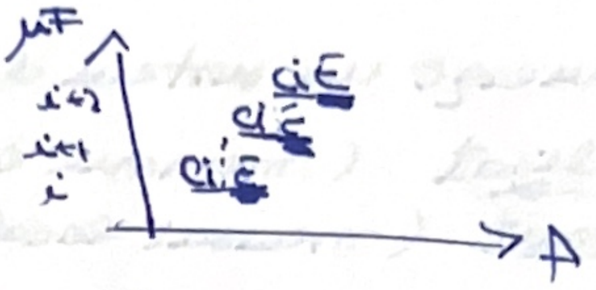
Microinstrucțiuni → operaționale (controlează primitivele funcționale)
→ neramificate / ramificate (respectă ordinea primitivelor)

Codificări: 1. Verticală (instrucțiuni, aplicate pe structuri specifice)
2. Orizontală (flexibilitate mare, paralelism maxim, volum ineficient de memorie)
3. Minimală (se grupează în clase de compatibilitate, la nivel de câmp - orizontal
între câmpuri - vertical)
4. Cu control regional (economie de memorie)
5. Cu control prin adresă (a se poate face simultan)
6. Mixtă (o microinstrucțiune împărțită în câmpuri, unele câmpuri controlează direct microoperații, altele specifică adresa)

Implementări: 1. Serie



2. Paralelă



3. Serie-paralel (combină ambele)